Studio Nx

JS 249 — 트랙 값 저장

2026-10-18

JavaScript 249 — 트랙 값 저장 (`updateTrack` · `syncTrackHead`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`
작업	S1-09 악기·이펙트 화면: 트랙 머리의 M · S · ⏻ 를 서버에 저장합니다 (FR-252)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	248 줄 아래 글 만들기 · 250 펼친 트랙의 칸 채우기

1. 하는 일

`updateTrack` 은 트랙 값 하나를 `PATCH /tracks/{id}` 로 보내고, 돌아온 값을 담아 둔 트랙 객체에 덮어씁니다. `syncTrackHead` 는 트랙 줄 머리의 ⏻ · M · S 버튼이 지금 눌린 상태를 읽어 그대로 저장합니다. 화면 모양은 arrange 쪽이 이미 바꾸므로, 이 함수는 **화면을 읽어 서버에 맞추는** 일만 합니다. `rowOf` 와 `pressed` 는 그때 쓰는 작은 도우미입니다.

```
M 클릭 → arrange 가 화면을 먼저 바꿈
        └ syncTrackHead(track)   aria-pressed 를 읽어
              └ updateTrack(track, {is_muted:true})   PATCH /tracks/4639
```

2. 언제 쓰나

쓰는 곳	
`sequencer-arrange.js`	트랙 머리 버튼을 누른 뒤 `syncTrackHead` 를 부릅니다
`saveDeviceModal` (261) · `toggleBypass` (261)	장치 값을 저장할 때 `updateDevice` 를 씁니다(249 의 트랙 판과 짝)
`syncTrackSubs` · `refreshCells`	줄을 찾을 때 `rowOf` 를 씁니다

3. 코드

```
/** 값 고치기 — PATCH /tracks/{id} (볼륨 · 팬 · M · S · 켜기) */
export async function updateTrack(track, patch) {
  const saved = await api.patch('/tracks/' + track.track_id, patch);
  Object.assign(track, saved);
  return saved;
}

/** 트랙 머리의 M · S · ⏻ 를 눌렀을 때 서버에 저장 (화면은 arrange 가 이미 바꿔 둠 → 다음 틱에 읽음) */
export function syncTrackHead(track) {
  const row = rowOf(track);
  if (!row || !active) return;
  const head = row.querySelector('.sq-track__head');
  if (!head) return;
  const patch = {
    is_enabled: pressed(head, '.sq-power'),
    is_muted: pressed(head, '.is-mute'),
  };
```

```
  if (head.querySelector('.is-solo')) patch.is_solo = pressed(head, '.is-solo');
  updateTrack(track, patch).catch(warn);
}

/** 트랙 줄 찾기 */
function rowOf(track) {
  return document.querySelector('.sq-track[data-track-id="' + track.track_id + '"]');
}

/** 버튼이 눌려 있나 */
function pressed(scope, sel) {
  const b = scope.querySelector(sel);
  return !!b && b.getAttribute('aria-pressed') === 'true';
}
```

문법 풀이

식	뜻
`api.patch('/tracks/' + track.track_id, patch)`	보낸 값만 바꾸는 요청입니다
`Object.assign(track, saved)`	서버가 돌려준 값으로 담아 둔 트랙을 갱신합니다
`if (!row || !active) return;`	데모 모드이거나 줄이 없으면 저장하지 않습니다
`head.querySelector('.is-solo')`	MASTER 나 RETURN 에는 솔로 버튼이 없어 있을 때만 넣습니다
`pressed(head, '.sq-power')`	`aria-pressed` 가 `'true'` 인지 봅니다
`.catch(warn)`	저장이 실패해도 경고만 남기고 화면은 그대로 둡니다

4. 왜 이렇게

선택	이유
화면을 먼저 바꾸고 저장	눌림 표시가 즉시 반응해야 손맛이 좋습니다. 서버 왕복을 기다리지 않습니다
저장 실패를 삼킴	잠깐 서버가 없어도 화면 조작은 이어져야 합니다
돌아온 값을 덮어씀	서버가 기본값을 채워 줄 수 있어 그 값을 기준으로 삼습니다
`updateDevice` 와 짝	트랙 값과 장치 값을 같은 방식으로 저장해 코드를 읽기 쉽게 합니다

5. 시험

시험 프로젝트의 Bass Synth M 을 눌러 `aria-pressed` 가 `true` 가 되고 `GET /tracks/4639` 의 `is_muted` 가 `true` 로 저장되는 것을 확인했습니다. 같은 트랙에서 다시 눌러 `false` 로 되돌렸습니다.